BURRA SEC

Passthrough Vaults Audit Report

Prepared by Burra Security

Version 1.0

Researchers

Filip
eeyore
Kiki

June 2, 2026

Contents

1 About Burra Security

Burra Security offers security auditing and advisory services with a special focus on cross-chain and interoperability protocols and their integrations. Learn about us at burrasec.com.

2 Disclaimer

A smart contract security review can never verify the complete absence of vulnerabilities. This is a time, resource, and expertise-bound effort where we try to find as many vulnerabilities as possible. We can not guarantee 100% security after the review or even if the review will find any vulnerabilities. Subsequent security reviews, bug bounty programs, and on-chain monitoring are recommended.

3 Risk Classification

	Impact: High	**Impact: Medium**	**Impact: Low**
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

4 Protocol Summary

Centrifuge Passthrough Vaults is a vault layer built on the Centrifuge protocol that routes user deposits directly into underlying Centrifuge pools without holding assets independently. The PassthroughVault contract acts as a transparent conduit, forwarding liquidity to the target pool while exposing a standard vault interface to depositors.

Asynchronous deposit and redemption requests are coordinated through an internal queue, implemented in QueueLib, which sequences pending operations and ensures orderly settlement against the underlying pool. This design accommodates the async nature of Centrifuge pool interactions behind a familiar entry point.

The vaults integrate with the ERC-4626 and ERC-7540 standards, enabling seamless composability with DeFi protocols and yield aggregators while preserving the full lifecycle management, queueing, execution, and settlement, required by Centrifuge's onchain asset management infrastructure.

5 Audit Scope

The following smart contracts were in the scope of the audit:

```
src/
├── libraries/
│   └── QueueLib.sol
└── PassthroughVault.sol
```

6 Executive Summary

Over the course of 3 days, the Burra Security team conducted an audit on the Passthrough Vaults smart contracts provided by Centrifuge. In this period, a total of 13 issues were found.

Summary

Project Name	Passthrough Vaults
Repository	passthrough-vaults
Commit	bbe0c9bd4959...
Audit Timeline	May 19th - May 21st
Methods	Manual Review

Issues Found

Critical Risk	0
High Risk	0
Medium Risk	0
Low Risk	5
Informational	8
Gas Optimizations	0
Total Issues	13

Summary of Findings

[L-1] `claimForAll` limitation allows for yield extraction	Resolved
[L-2] Passthrough omits request cancellation	Resolved
[L-3] Cumulative deposit settled amount can be understated after rounded claims	Resolved
[L-4] Cumulative redeem settled amount can be understated after rounded claims	Resolved
[L-5] Restricted users can interact with the underlying vault through the passthrough vault.	Acknowledged
[I-1] Spot convertTo views diverge from async claim outcomes	Resolved
[I-2] Redundant `permissioned` check in `PassthroughVault::deposit(2-arg)`	Resolved
[I-3] `@inheritdoc` references functions without Natspec in interface	Resolved
[I-4] Cursor invariant is silently breakable by a Centrifuge governance spell	Resolved
[I-5] Memberlist revocation between request and claim freezes the user's queued position with no on-chain exit	Resolved
[I-6] Permissionless factory enables honeypot deployments	Resolved
[I-7] Shared hub-side `UserOrder` row forces all PV users into the queued slot during the approve-notify window	Resolved
[I-8] Centrifuge vault and share-class administration can silently disable the PassthroughVault	Resolved

7 Findings

7.1 Low Risk

7.1.1 `claimForAll` limitation allows for yield extraction

Target: PassthroughVault.sol

Severity:

- Impact: Low
- Likelihood: Medium

Description: `PassthroughVault` routes all underlying async deposit and redeem activity through address this, so `AsyncRequestManager` keeps one shared `redeemPrice` and `depositPrice` for every passthrough user. The local FIFO queue only tracks how much each controller may claim. It does not store the fulfillment rate from each user's own epoch.

Each async flow has two distinct phases. A request is pending until the hub fulfills it. After fulfillment the position is claimable, but assets or shares are not delivered until the user calls deposit or redeem. Between fulfillment and claim, the fulfilled amount stays inside the shared underlying bucket and is included in the next blended price calculation when another user is fulfilled.

When `claimForAll` is false, only the controller may claim. No third party can force a settled position out of the shared bucket before a later user claims at the blended rate. Resulting in there being an incentive for users to not finalize deposits and request and instead leach off future users.

Redeem example:

- Alice requests 100 shares and is fulfilled for 100 assets at 1 asset per share.
- She does not claim. However, if she claimed immediately she would receive **100 assets**.
- Share value rises. Bob requests 100 shares and is fulfilled for 200 assets at 2 assets per share.
- The underlying bucket now blends both fulfillments to 1.5 assets per share.
- Bob claims 100 shares and receives 150 assets instead of 200.
- Alice then claims 100 shares and receives **150 assets** instead of 100.
- Alice **gains 50 assets** by waiting. Bob loses 50 assets versus his own fulfillment.

The deposit path mirrors this with shares instead of assets when an earlier fulfilled deposit stays unclaimed while later depositors enter at a better rate.

Recommended Mitigation: Consider restricting `claimForAll` to be always true for async vaults, or drop `claimForAll`. This would prevent bad actors from forcing users into subsidizing their gains and losses.

Client: Documented at commit `21138c0ed0448ad198b10faabf26662d84a1ebc9`

BurraSec: Documented as recommended.

7.1.2 Passthrough omits request cancellation

Target: PassthroughVault.sol

Severity:

- Impact: Low
- Likelihood: Low

Description: `PassthroughVault` exposes `requestDeposit`, `requestRedeem`, `deposit`, and `redeem`, but it does not expose the underlying async vault cancellation flows. The underlying `AsyncVault` and `BaseAsyncRedeemVault` support `cancelDepositRequest`, `cancelRedeemRequest`, and cancel claim functions through `AsyncRequestManager`, but `PassthroughVault` users cannot reach those functions after entering through the wrapper. This traps passthrough users in pending async deposit and redeem requests until the underlying request is fulfilled or cancelled externally by protocol operations. A user who submits through `PassthroughVault` loses the cancellation protection available to direct vault users.

In situations where configurations are changed (reserve cap is lowered), fulfillment is slow, and/or the request queue is exceptionally long a user will not be able to access their funds for a prolonged period of time. All while price can continue to move and the user will complete their deposit or redeem at a worse than expected price.

Recommended Mitigation: Clearly document that deposit and redeem actions through the PassthroughVault are not reversible.

Client: Fixed at commit `c22e4fab6d5e2ae7a6217b9c36812eb3ff5c7041`

BurraSec: Documented as recommended

7.1.3 Cumulative deposit settled amount can be understated after rounded claims

Target: PassthroughVault.sol

Severity:

- Impact: Low
- Likelihood: High

Description: `_getCumulativeDepositSettled()` derives the passthrough's cumulative settled deposit amount from `vault.maxDeposit(address(this)) + totalDepositClaimed`. This value can be understated because the underlying Centrifuge `AsyncVault` claim path, implemented through `AsyncRequestManager.deposit()`, uses asymmetric rounding: it debits `state.maxMint` by `ceil(assets / price)` shares, but transfers only `floor(assets / price)` shares to the receiver.

Whenever a claim amount is not exactly aligned with the current deposit price, the underlying `maxDeposit` can decrease by slightly more than the passthrough's `totalDepositClaimed` increases. As a result, `_getCumulativeDepositSettled()` can report less than the true fulfilled deposit amount.

The practical consequence is concentrated at the back of the FIFO queue. Earlier users can still consume the underlying claimable balance, but the accumulated rounding gap is pushed to the last depositor in the settled range. That last depositor may be left with a small `pendingDepositRequest` amount that is no longer claimable because the underlying `vault.maxDeposit(address(this))` has already been exhausted. If no further deposit fulfillment arrives, this tail remainder stays stuck.

The economic value is expected to be very small because each drift event is bounded by less than one share worth of assets. The issue is mainly an accounting/UX problem: the final queued depositor can see non-zero pending assets that cannot be claimed.

Recommended Mitigation: Acknowledge this as a documented limitation of the passthrough design. The README should state that Centrifuge `AsyncVault.deposit()` claims can leave a tiny rounding remainder assigned to the last depositor in the settled queue, and that this dust may remain pending if no further fulfillment arrives. Fully eliminating the behavior would require a broader redesign of the claim accounting, for example around share-based `mint()` claims.

Client: Fixed at commit 11a5a3c613afddb6d6d139bad9b1ccdaeb7dc959

BurraSec: Documented as recommended.

7.1.4 Cumulative redeem settled amount can be understated after rounded claims

Target: PassthroughVault.sol

Severity:

- Impact: Low
- Likelihood: High

Description: `_getCumulativeRedeemSettled()` derives the passthrough's cumulative settled redeem amount from `vault.maxRedeem(address(this)) + totalRedeemClaimed`. This value can be understated because the underlying Centrifuge `AsyncVault` redeem-claim path, implemented through `AsyncRequestManager.redeem()`, uses asymmetric rounding: it debits `state.maxWithdraw` by `ceil(shares * redeemPrice)` assets, but transfers only `floor(shares * redeemPrice)` assets to the receiver.

When a redeem claim is not exactly aligned with the current redeem price, `state.maxWithdraw` is reduced by slightly more assets than the user receives. The next `vault.maxRedeem(address(this))` converts the remaining asset balance back into shares with rounding down, so `_getCumulativeRedeemSettled()` can report less than the true fulfilled redeem amount.

The practical consequence is concentrated at the back of the redeem FIFO queue. Earlier users can consume the available redemption balance, but the accumulated rounding gap is pushed to the last redeemer in the settled range. That user may be left with a small `pendingRedeemRequest` amount that is no longer claimable because the underlying `vault.maxRedeem(address(this))` has already been exhausted. If no further redeem fulfillment arrives, this tail remainder stays stuck.

This is more meaningful than the deposit-side version because the donated dust is asset-denominated. For common configurations with a 6-decimal asset and 18-decimal shares, redeem dust accumulates in asset wei, while deposit dust accumulates in share wei. The absolute value is still small, but the user-visible effect is a non-zero pending redeem balance that cannot currently be claimed.

Recommended Mitigation: Acknowledge this as a documented limitation of the passthrough design. The README should state that Centrifuge `AsyncVault.redeem()` claims can leave a tiny asset-denominated rounding remainder assigned to the last redeemer in the settled queue, and that this dust may remain pending if no further fulfillment arrives. Fully eliminating the behavior would require a broader redesign of the claim accounting, for example around asset-based `withdraw()` claims.

Client: Fixed at commit 11a5a3c613afddb6d6d139bad9b1ccdaeb7dc959

BurraSec: Documented as recommended.

7.1.5 Restricted users can interact with the underlying vault through the passthrough vault.

Target: PassthroughVault.sol

Severity:

- Impact: Low
- Likelihood: Medium

Description: Because the underlying vault interprets the caller as the passthrough vault any user who is placed under any of Centrifuge's restrictions can bypass said restrictions through a passthrough vault. This allows any user to freely request deposits, and because claims are initially minted to the passthrough vault the mint will pass the check as well. For the claim process the user can use a unblocked receiver and obtain the share tokens.

PassthroughVaults are permissionless to deploy meaning any user who is blocked from Centrifuge can deploy a Passthroughvault for themselves and access functionality that they otherwise would not be able to. Additionally

some passthrough vaults may be permissionless in nature with no `memberlist` set. Allowing the same outcome of prohibited users interacting with the underlying vaults.

Recommended Mitigation: Validate that the caller of the passthrough vault as well as the controller and receiver are not prohibited from interacting with the underlying vault.

Client: Acknowledged.

BurraSec: Acknowledged.

7.2 Informational

7.2.1 Spot convertTo views diverge from async claim outcomes

Target: PassthroughVault.sol

Severity: Informational

Description: The `PassthroughVault` `convertToShares` and `convertToAssets` functions forward directly to the underlying vault conversion functions. The underlying `BaseVaults` conversion functions use the current Centrifuge epoch price, so they expose a spot conversion rather than the settled async claim price used by the controller state. Async passthrough claims use different pricing.

Because `PassthroughVault` forwards all users through `address(this)` as the underlying controller, those stored prices are blended across all claimable amounts for the wrapper. The interface does not document this distinction. Integrators reading `convertToShares` or `convertToAssets` as a claim preview receive values that differ from the shares or assets returned by `deposit` or `redeem` whenever the settled controller price differs from the spot price.

Recommended Mitigation: Document `convertToShares` and `convertToAssets` as spot conversions which will differ from the blended price that is actually used for deposits or redeems, and should not be used for such calculations.

Client: Fixed at commit `cf820b898059f66e5951e1ebc57d2e802d30577c`

BurraSec: Documented as recommended

7.2.2 Redundant `permissioned` check in `PassthroughVault::deposit(2-arg)`

Target: PassthroughVault.sol#L94

Severity: Informational

Description: The two-argument `deposit(uint256 assets, address receiver)` at `PassthroughVault.sol:94` already checks `permissioned(msg.sender)` before forwarding to the three-argument overload with `controller = msg.sender`. Since the three-argument version at `PassthroughVault.sol:99` performs the same `permissioned(controller)` check again, every call through the two-argument path triggers two identical memberlist lookups. As `isPermissioned` is an external call, this adds unnecessary overhead to every sync deposit and async claim made via the two-argument function.

Recommended Mitigation: Remove the `permissioned` modifier from the two-argument function. The check is fully covered by the three-argument function it delegates to.

```
- function deposit(uint256 assets, address receiver) external permissioned(msg.sender) returns (uint256
↪  shares) {
+ function deposit(uint256 assets, address receiver) external returns (uint256 shares) {
      shares = deposit(assets, receiver, msg.sender);
  }
```

Client: Fixed at commit `bf9cb434aecf8bf2f26c43f8b4e7cd9af8582a8c`

BurraSec: Documented as recommended

7.2.3 `@inheritdoc` references functions without Natspec in interface

Target: PassthroughVault.sol#L251-L264, IPassthroughVault.sol#L110-L112

Severity: Informational

Description: Three functions in `PassthroughVault.sol` carry `/// @inheritdoc IPassthroughVault` comments but the corresponding declarations in `IPassthroughVault.sol` have no natspec at all. The affected functions are `totalAssets()` at `IPassthroughVault.sol:110`, `convertToShares(uint256)` at `IPassthroughVault.sol:111`, and `convertToAssets(uint256)` at `IPassthroughVault.sol:112`. The `@inheritdoc` tag silently fails to inherit anything, giving the reader a false sense that documentation exists somewhere in the inheritance chain.

Recommended Mitigation: Add comments to the three undocumented interface functions. The descriptions can be brief since the semantics are inherited from ERC-4626.

Client: Fixed at commit `61a7b3e1ccd0ff2307442be24a764425f02f89cb`

BurraSec: Documented as recommended

7.2.4 Cursor invariant is silently breakable by a Centrifuge governance spell

Target: PassthroughVault.sol

Severity: Informational

Description: The passthrough's FIFO queue depends on the monotonicity of two computed values:

```
function _getCumulativeDepositSettled() internal view returns (uint128) {
    return vault.maxDeposit(address(this)).toUint128() + totalDepositClaimed;
}

function _getCumulativeRedeemSettled() internal view returns (uint128) {
    return vault.maxRedeem(address(this)).toUint128() + totalRedeemClaimed;
}
```

Both depend on the underlying's `state.maxMint` and `state.maxWithdraw` for the passthrough's controller row in `AsyncRequestManager.investments`. These storage slots are mutable by any holder of `auth` (ward) on the manager. Through Centrifuge's standard governance flow (`ProtocolGuardian` schedules a spell, 48h timelock on mainnet, `Root.executeScheduledRely` grants the spell temporary ward access, spell executes, ward revoked), a spell can call:

- `AsyncRequestManager.deposit(vault, X, anyReceiver, controller=passthrough)` to debit `state.maxMint` and divert shares to a non-passthrough receiver. `vault.maxDeposit(passthrough)` regresses; `totalDepositClaimed` is unchanged. The passthrough's settled cursor silently drops.
- `AsyncRequestManager.redeem(vault, X, anyReceiver, controller=passthrough)` to debit `state.maxWithdraw` and divert assets. `vault.maxRedeem(passthrough)` regresses; `totalRedeemClaimed` is unchanged. The redeem cursor silently drops.
- `AsyncRequestManager.cancelDepositRequest(vault, passthrough, _)` to set `pendingCancelDepositRequest = true` on the passthrough's row. The hub then routes the cancelled assets into `claimableCancelDepositRequest`. The passthrough exposes no `claimCancel*` function and cannot designate an operator, so the cancelled balance is permanently stranded in the underlying.
- `AsyncRequestManager.cancelRedeemRequest(vault, passthrough, _)` applies analogously. This case is strictly worse than the deposit cancel: the cancelled shares were originally users' shares deposited into the redeem queue, and they end up locked in `claimableCancelRedeemRequest` with no passthrough-side path to release them back to users.

In every case, the PassthroughVault has no on-chain visibility into the change.

The protocol's normal trust posture is that Centrifuge governance does not act adversarially: spells exist for legitimate emergency-recovery reasons (pool wind-down, token recovery, frozen-fund remediation) and pass through a multisig + 48h public timelock.

The issue is reported as informational, on the grounds that the PassthroughVault's marketing posture (the `README` calling the contract "fully immutable: no admin functions, no upgrades, no escape hatch", and the contract NatSpec saying "Contract is fully immutable: no admin, no upgrades, no escape hatch") is binary-true of `PassthroughVault.sol` itself but is silent on the fact that 100% of the economic state lives in `AsyncRequestManager` storage and is mutable by Centrifuge governance.

Recommended Mitigation: Update the README and NatSpec with a short trust-model paragraph.

Client: Fixed at commit `668bf3e51b84b9e6585f64cd75b155b67b0f7bf9`

BurraSec: Documented as recommended

7.2.5 Memberlist revocation between request and claim freezes the user's queued position with no on-chain exit

Target: PassthroughVault.sol

Severity: Informational

Description: Every state-mutating function uses `permissioned(controller)`, including the claim paths (`deposit(uint256,address)`, `deposit(uint256,address,address)`, `redeem(uint256,address,address)`). If the memberlist owner revokes a controller between their `requestDeposit` / `requestRedeem` and the moment they would claim, the modifier reverts with `NotMember()` on every subsequent claim attempt. The owner can also modify the logic within the `isPermissioned` function to revert and impact all members. A `claimForAll` keeper cannot rescue them either: the modifier still checks `permissioned(controller)`. Their queued shares or assets sit indefinitely in the underlying's `PoolEscrow` under the passthrough's name until the memberlist re-admits them.

A secondary effect on the redeem flow: the revoked user's stuck position keeps contributing to the blended `state.redeemPrice` on every subsequent hub fulfillment. Future redeemers therefore receive a price that is partially set by the stuck position, instead of fully reflecting their own epoch. The effect is bounded by the revoked user's share of the total queue and shrinks as new fulfillments arrive, but never goes to zero until re-admission.

The protocol's intended trust model is that the memberlist owner is the policy authority and "revoke = freeze claims" is a defensible, deliberate policy lever. The issue is that this policy is not documented anywhere: the README's access-control section mentions only that membership is checked on the controller, and does not state that revocation freezes already-settled claims.

Recommended Mitigation: Either:

1. Move the `permissioned(controller)` check off the claim paths (`deposit(2-arg)`, `deposit(3-arg)`, `redeem`) while keeping it on the request paths (`requestDeposit`, `requestRedeem`). This permits a user to claim a position they had already validly queued before revocation. The memberlist still controls who can submit new requests; it loses the ability to silently freeze in-flight positions. Or,
2. Keep the current policy and document it explicitly in the README's access-control section: that the memberlist owner has authority to freeze any in-flight claim by revocation, that there is no on-chain alternative exit, and that a revoked user's stuck position continues to dilute the price-blending exposure of all other passthrough users until re-admission.

Client: Documented at commit `5f6464f84f3390b1110f135536f8b1cfdbf7aa4c`

BurraSec: Behavior was documented in `README.md`.

7.2.6 Permissionless factory enables honeypot deployments

Target: PassthroughVault.sol

Severity: Informational

Description: `PassthroughVaultFactory.newVault` accepts any `vault` address with no validation that it is a legitimate Centrifuge vault. The constructor only reads `asset()` and `share()` from the supplied address.

An attacker can deploy an `IUnderlyingVault`-shaped malicious contract, then call `factory.newVault(maliciousVault, ...)`. The resulting PassthroughVault is deployed from the same audited bytecode, has a deterministic CREATE2 address, is immutable, and has no admin. On-chain it is indistinguishable from a legitimate PassthroughVault. When a user calls `requestDeposit`, their asset is forwarded to the attacker's underlying.

Recommended Mitigation: Consider documenting in the README that the factory is permissionless and that passthrough legitimacy depends on independent verification of the wrapped `vault` address.

Client: Fixed at commit `156a908d6eff9b8abc70bb9ddcc1c0fd38e8ade6`

BurraSec: Documented as recommended.

7.2.7 Shared hub-side `UserOrder` row forces all PV users into the queued slot during the approve-notify window

Target: PassthroughVault.sol

Severity: Informational

Description: The PassthroughVault is the sole `investor` key for the entire user base in the hub-side `BatchRequestManager.depositRequest[poolId][scId][assetId][passthrough]` and the symmetric redeem mapping. All PassthroughVault users share one `UserOrder { pending, lastUpdate }` and one `QueuedOrder { amount, isCancelling }` row.

The hub gates whether a new request is written to `pending` or to `queued` via `_canMutatePending`:

```
return currentEpoch <= 1 || userOrder.pending == 0 || userOrder.lastUpdate >= currentEpoch;
```

`lastUpdate` is advanced only inside `_claimDeposit` (called by `notifyDeposit`), not by `approveDeposits` or `issueShares`. So during the window between `approveDeposits` (which advances `currentEpoch`) and `notifyDeposit(PV)` (which advances `userOrder.lastUpdate`), the check evaluates to `false` and every new request lands in `queued` instead of `pending`. The queued amount is not approved in subsequent `approveDeposits` cycles until a later `notifyDeposit(PV)` flushes it. Each affected user waits at least one extra epoch.

This is the same shared-row structural property as the price blending documented in the README's "Price mechanics" section, surfaced on a different slot. The PassthroughVault pools N investors into a single hub-side row; the underlying treats them as one user; mechanics that would be self-grief for a direct user become other-grief for everyone else in the PassthroughVault.

Two manifestations:

1. **Normal operation.** Whenever the operator's flow has any gap between `approveDeposits` + `issueShares` and `notifyDeposit(PV)` (typical for cross-chain pools, or pools where notify is batched), every new request during that window is queued. Users routinely lose an epoch with no malicious actor present.
2. **Adversarial amplification.** An attacker calls `requestDeposit(1, ...)` to ensure `userOrder.pending > 0` and `lastUpdate < currentEpoch`. As long as the operator runs `approve` before `notify`, the queued window stays dirty across cycles, and every other PV user is permanently shifted into the +1 epoch path. Cost: 1 wei + gas, repeatable per cycle.

The redeem side has the symmetric mechanism via `redeemRequest[..][PV]`. The user experience is worse there: the delayed user's shares are already in PoolEscrow during the extra epoch, exposed to ongoing `state.redeemPrice` re-blending while they wait.

The PassthroughVault has no on-chain mechanism to detect the state or force a `notifyDeposit`. The shared-row property is intrinsic to the single-controller design. There is no fund theft; the worst-case harm is bounded to an extra epoch delay per affected user.

Recommended Mitigation: Document the shared-row exposure in the PV's README alongside the existing "Price mechanics" section.

Client: Fixed at commit `21138c0ed0448ad198b10faabf26662d84a1ebc9`

BurraSec: Documented as recommended

7.2.8 Centrifuge vault and share-class administration can silently disable the PassthroughVault

Target: PassthroughVault.sol

Severity: Informational

Description: The PassthroughVault pins its `vault` pointer as `immutable` and reads aggregate state (`vault.maxDeposit(this)`, `vault.maxRedeem(this)`) from whatever `manager` and transfer hook are currently configured by Centrifuge on that vault. Several legitimate Centrifuge governance and pool-operator actions break the PassthroughVault's queue without on-chain detection or recovery.

In every case the PassthroughVault's `settled` value collapses to `totalDepositClaimed` (or `totalRedeemClaimed`), claims revert, and any subsequent non-zero fulfillment is absorbed via FIFO by the earliest-queued investor regardless of which epoch settled their request.

Sub-cases:

1. `validUntil[PV]` **lapse on the share-class transfer hook.** Membership has a `uint64 validUntil` expiry. Once `block.timestamp > validUntil[PV]`, the hook's `isTargetMember(PV)` returns false. `AsyncRequestManager.maxDeposit` and `maxRedeem` return 0. All flows revert. Trigger: pool operator (`manager[token]` role) sets a finite expiry and lets it pass without renewal.
2. `freeze(token, PV)`**.** Hook's `isSourceOrTargetFrozen` returns true for any transfer involving the PassthroughVault. Same failure mode. Trigger: pool operator with the manager role.
3. **Hook identity swap to** `FullRestrictions`**.** The PassthroughVault's deposit-claim path is a two-hop pattern (`vault.deposit(assets, this, this)` then `safeTransfer(share, receiver)`), which only works when the hook's fallback for arbitrary transfers returns true (`FreelyTransferable`, `FreezeOnly`, `RedemptionRestrictions`). Under `FullRestrictions` the fallback returns `isTargetMember(receiver)`, which fails for any non-member end user. Trigger: pool operator runs `hub.updateShareHook` to switch to `FullRestrictions`. Strands every in-flight settled deposit.
4. `unlinkVault`**.** `AsyncRequestManager` claim entry points start with `_checkIsLinked(vault_)`, which reverts `VaultNotLinked` if `vaultRegistry.isLinked(vault) == false`. The PassthroughVault's vault pointer is immutable, so it cannot rebind to a successor vault after a migration. Trigger: pool operator runs `hub.updateVault(..., VaultUpdateKind.Unlink)` as part of any vault migration (Sync↔Async swap, factory replacement, etc.).
5. **Manager swap without** `investments` **state migration.** `vault.file("manager", newARM)` repoints the vault to a fresh `AsyncRequestManager` whose `investments` mapping is empty. The PassthroughVault's existing per-controller row on the old manager is abandoned. `vault.maxDeposit(this)` returns 0 thereafter. Trigger: Centrifuge governance via a spell upgrading the manager set without migrating per-controller state. Precedent exists in production migration spells.

In all five cases the PassthroughVault has zero on-chain visibility, zero detection, and zero recovery. The damage is bounded to a DoS on existing positions plus FIFO-driven redistribution of any subsequent fulfillments to the queue head; there is no fund theft and the underlying's own accounting remains internally consistent. Funds in `PoolEscrow` remain attributable to the PassthroughVault's controller row, recoverable only by additional administrative action from the same authority that triggered the break.

Under the standard audit trust model (Centrifuge governance and pool operators act in good faith), all five sub-cases reduce to operational risk.

Recommended Mitigation: Document in the README that the PassthroughVault inherits its operational availability from Centrifuge's vault lifecycle and share-class administration.

Client: Fixed at commit `668bf3e51b84b9e6585f64cd75b155b67b0f7bf9`

BurraSec: Documented as recommended.